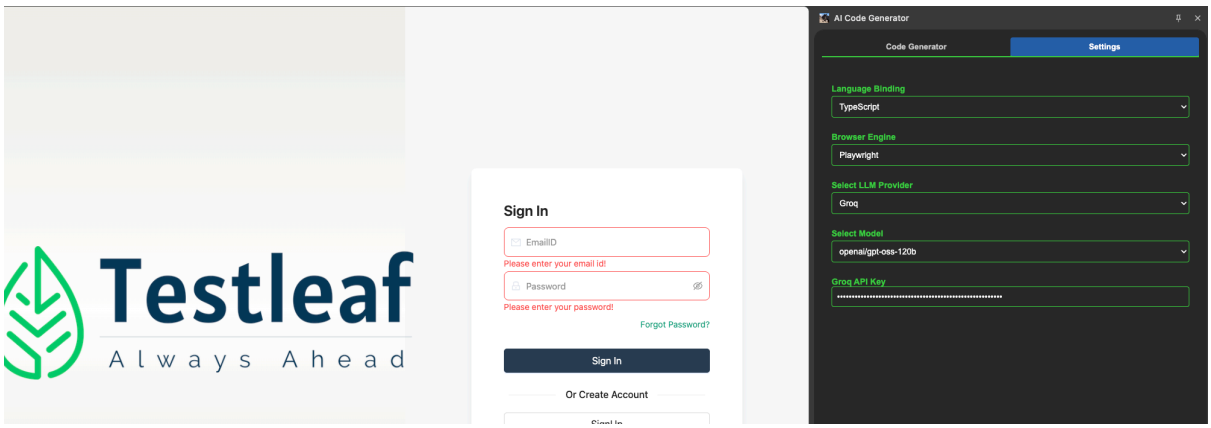
Week03 – Day01 Assignment Answers

Assignment description:

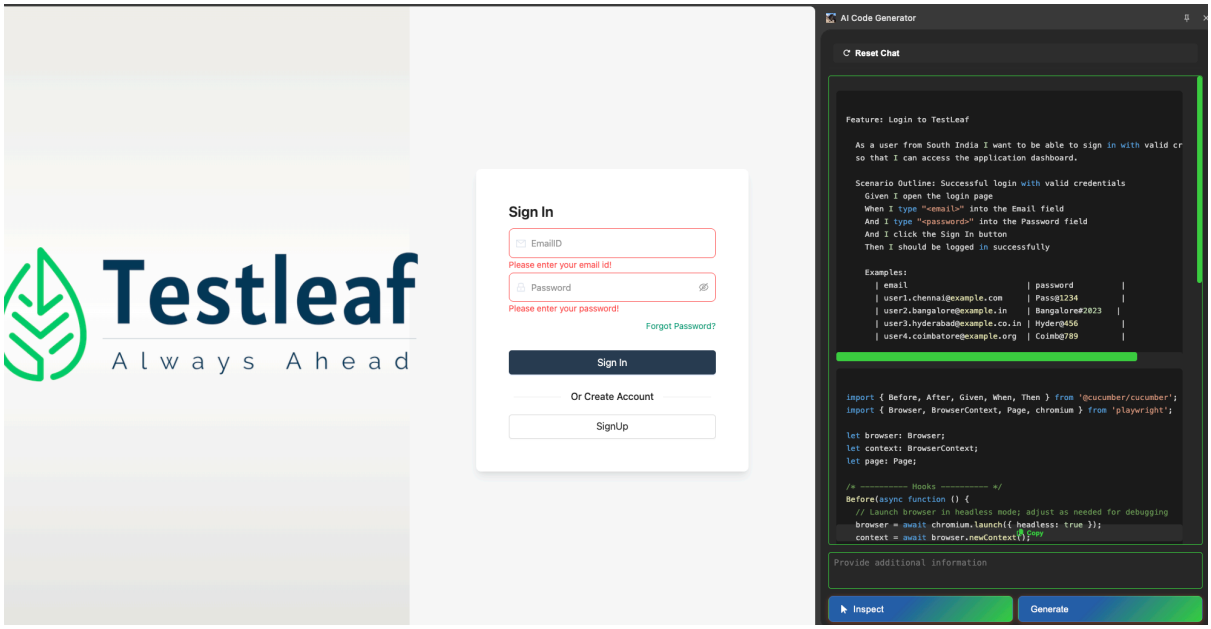
Implement playwright/typescript with vibe coding

URL Used: https://api.testleaf.com/login

Settings Screenshot:



URL Screenshot:

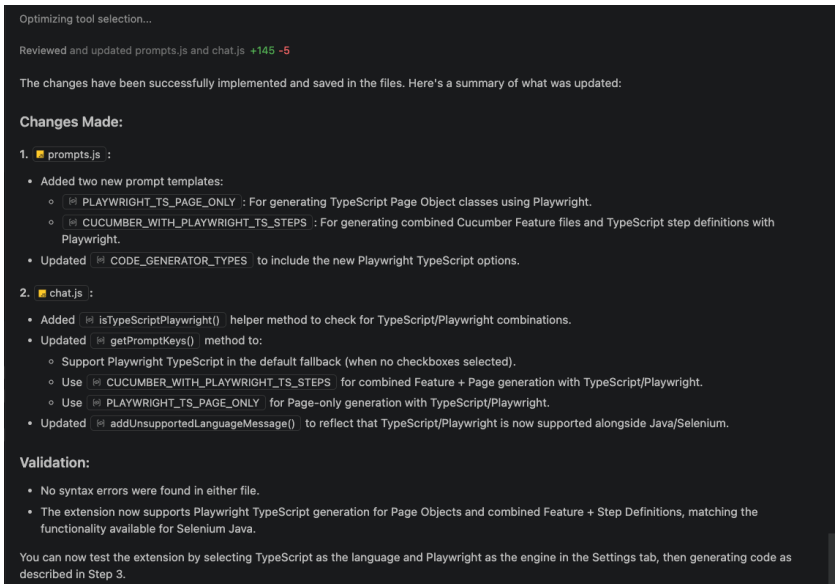


Playwright/TS output from extension:

```
Feature: Login to TestLeaf

  As a user from South India I want to be able to sign in with valid
credentials
  so that I can access the application dashboard.

  Scenario Outline: Successful login with valid credentials
    Given I open the login page
    When I type "<email>" into the Email field
    And I type "<password>" into the Password field
    And I click the Sign In button
    Then I should be logged in successfully

    Examples:
      | email                          | password          |
      | user1.chennai@example.com      | Pass@1234         |
      | user2.bangalore@example.in     | Bangalore#2023   |
      | user3.hyderabad@example.co.in | Hyder@456         |
      | user4.coimbatore@example.org   | Coimb@789         |

import { Before, After, Given, When, Then } from '@cucumber/cucumber';
import { Browser, BrowserContext, Page, chromium } from 'playwright';

let browser: Browser;
let context: BrowserContext;
let page: Page;

/* ---------- Hooks ---------- */
Before(async function () {
  // Launch browser in headless mode; adjust as needed for debugging
  browser = await chromium.launch({ headless: true });
  context = await browser.newContext();
  page = await context.newPage();
});

After(async function () {
  await page.close();
  await context.close();
  await browser.close();
});

/* ---------- Step definitions ---------- */
Given('I open the login page', async function () {
  await page.goto('https://api.testleaf.com/login', { waitUntil:
'networkidle' });
});

When('I type {string} into the Email field', async function (email:
string) {
  const emailInput = page.locator('#loginForm_email');
  await emailInput.waitFor({ state: 'visible', timeout: 5000 });
```

```
  await emailInput.fill(email);
});

When('I type {string} into the Password field', async function (password:
string) {
  const passwordInput = page.locator('#loginForm_password');
  await passwordInput.waitFor({ state: 'visible', timeout: 5000 });
  await passwordInput.fill(password);
});

When('I click the Sign In button', async function () {
  const signInButton = page.locator('button.login-button');
  await signInButton.waitFor({ state: 'attached', timeout: 5000 });

  // Click and wait for navigation or a post-login element
  await Promise.all([
    page.waitForNavigation({ waitUntil: 'networkidle' }),
    signInButton.click(),
  ]);
});

Then('I should be logged in successfully', async function () {
  // Example verification: dashboard card appears after a successful
login
  const dashboardHeader = page.locator('.ant-card-body:has-
text("Dashboard")');
  await dashboardHeader.waitFor({ state: 'visible', timeout: 10000 });

  // Optional: also assert URL contains expected path
  await page.waitForURL(/.*\/dashboard/, { timeout: 5000 });
});
```

VS Code agent screenshot:



Optimizing tool selection...

Reviewed and updated prompts.js and chat.js +145 -5

The changes have been successfully implemented and saved in the files. Here's a summary of what was updated:

Changes Made:

1. prompts.js :
   - Added two new prompt templates:
     - PLAYWRIGHT_TS_PAGE_ONLY : For generating TypeScript Page Object classes using Playwright.
     - CUCUMBER_WITH_PLAYWRIGHT_TS_STEPS : For generating combined Cucumber Feature files and TypeScript step definitions with Playwright.
   - Updated CODE_GENERATOR_TYPES to include the new Playwright TypeScript options.
2. chat.js :
   - Added isTypeScriptPlaywright() helper method to check for TypeScript/Playwright combinations.
   - Updated getPromptKeys() method to:
     - Support Playwright TypeScript in the default fallback (when no checkboxes selected).
     - Use CUCUMBER_WITH_PLAYWRIGHT_TS_STEPS for combined Feature + Page generation with TypeScript/Playwright.
     - Use PLAYWRIGHT_TS_PAGE_ONLY for Page-only generation with TypeScript/Playwright.
   - Updated addUnsupportedLanguageMessage() to reflect that TypeScript/Playwright is now supported alongside Java/Selenium.

Validation:

- No syntax errors were found in either file.
- The extension now supports Playwright TypeScript generation for Page Objects and combined Feature + Step Definitions, matching the functionality available for Selenium Java.

You can now test the extension by selecting TypeScript as the language and Playwright as the engine in the Settings tab, then generating code as described in Step 3.